

AMAZON STREAMING PROJESİ: UÇTAN UCA TEKNİK ANALİZ VE UYGULAMA

Özkan Yılmaz 220201040

Umut Kahraman 220201108

Furkan Can İşci 220201125

Ömer Bayrak 220201120

1. GİRİŞ

Bu proje, modern büyük veri ekosisteminde verinin ham halden (raw) iş değerine (insight) dönüşme sürecini kapsayan karmaşık bir mühendislik çalışmasıdır. Amazon Mobile Electronics veri seti üzerinden kurgulanan bu sistem, sadece statik bir veri analizi değil, saniyeler içinde binlerce veriyi işleyebilen canlı bir "Veri Fabrikası" (Data Factory) niteliğindedir. Projenin ana omurgasını, endüstride "Medallion Architecture" olarak bilinen katmanlı yapı oluşturur. Bu yapı sayesinde veri; Bronze katmanında ham haliyle korunur, Silver katmanında teknik hatalardan arındırılır ve Gold katmanında makine öğrenmesi modelleri için "altın standartta" bir öznitelik tablosuna dönüştürülür.

Sistemin temel taşı Docker Konteynerizasyonudur. Docker Compose aracılığıyla ZooKeeper, Kafka, Spark Master, Spark Worker ve Jupyter servisleri birbirine entegre edilmiştir. Bu mikroservis mimarisi, projenin herhangi bir donanım bağımlılığı olmadan her ortamda aynı performansı göstermesini sağlar. Özellikle yerel makinelerde karşılaşılan disk alanı ve bellek kısıtları, Docker'ın kaynak yönetimi ve Spark'ın dağıtık hesaplama gücü ile optimize edilmiştir.

2. VERİ AKIŞI VE ALTYAPI MİMARİSİ (BRONZE KATMANI)

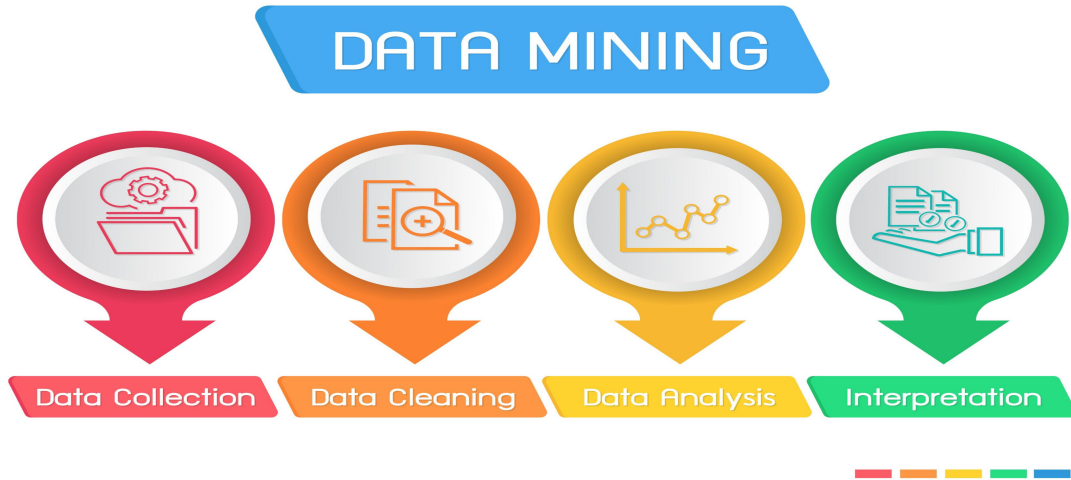
Projenin ilk kritik adımı, yaklaşık 55 GB boyutundaki TSV dosyasının sisteme enjekte edilmesidir. Bu işlem için Python tabanlı bir Kafka Producer (kafka_producer.py) geliştirilmiştir. Producer, veriyi satır satır okuyarak JSON formatına dönüştürür ve amazon_reviews adlı Kafka Topic'ine basar. Burada kullanılan acks='all' konfigürasyonu, her mesajın Kafka brokerları tarafından onaylandığını garanti altına alarak veri kaybını sıfıra indirir. GZIP sıkıştırma tekniği ile ağ üzerindeki yük azaltılmış, saniyede yüzlerce yorumun akışı sağlanmıştır.

Kafka, bu mimaride bir tampon (buffer) görevi görür. Veriyi üreten yapı (Producer) ile işleyen yapı (Spark) birbirinden tamamen ayrılmıştır (Loose Coupling). Bu sayede Spark tarafında bir gecikme yaşansa bile Kafka veriyi diskinde tutmaya devam eder ve sistemin hata toleransı artırılmış olur. ZooKeeper ise bu karmaşık trafik içinde Kafka'nın lider seçimlerini ve metadata yönetimini koordine eder.

3. VERİ RAFİNERİSİ VE TEMİZLEME (SILVER KATMANI)

Kafka topic'inden akan veriler, Spark Structured Streaming (streaming.py) tarafından yakalanır. Bu aşama, ham verinin teknik olarak "temiz" hale getirildiği Silver katmanıdır. Spark, Kafka'dan gelen her bir mesajı bir şema (schema) süzgecinden geçirir. from_json fonksiyonu ile string haldeki veriler; star_rating için integer, review_date için date formatına dönüştürülür.

Veri kalitesini korumak adına, modelleme için hayati önem taşıyan review_id, product_id ve star_rating gibi alanları boş (null) olan kayıtlar dropna fonksiyonu ile sistemden atılır. Temizlenen bu veriler, hızlı okuma ve yüksek sıkıştırma performansı sunan Parquet formatında diske yazılır. Hemen ardından devreye giren Keşifsel Veri Analizi (EDA) süreci, verinin istatistiksel profilini çıkarır. Bu analizlerde, Amazon kullanıcılarının çoğunlukla 5 yıldız vermeye eğilimli olduğu (skewness) ve 55 GB'lık verinin zamansal dağılımındaki trendler net bir şekilde ortaya konur.



4. ÖZELLİK MÜHENDİSLİĞİ: VERİYE ZEKA KATMA (GOLD KATMANI)

Silver katmanındaki temiz veri, makine öğrenmesi modelleri için yeterince anlamlı değildir. Bu noktada projenin en özgün kısmı olan Feature Engineering (feature_engineering.py) başlar. Ham veriden 5 ana öznitelik türetilmiştir:

review_word_count: Yorum metnindeki kelime sayısı. Uzun yorumlar genellikle daha fazla detay ve ürün kalitesine dair derin ipuçları taşır.

helpfulness_ratio: helpful_votes değerinin total_votes değerine oranı. Bu, sosyal bir onay mekanizmasıdır; diğer kullanıcıların faydalı bulduğu yorumlar model için daha güçlü bir sinyaldir.

headline_length: Başlıktaki karakter sayısı. Kullanıcının vurgu şiddetini temsil eder.

is_verified: verified_purchase bilgisinin 1 ve 0'a dönüştürülmesi. Ürünü gerçekten satın alanın yorumu, modelin güven puanını artırır.

is_long_review: Metin uzunluğunun veri seti ortalamasıyla kıyaslandığı bir flag alanıdır.

Bu özellikler Spark SQL fonksiyonları kullanılarak dağıtık bir şekilde hesaplanır ve ACID (Atomicity, Consistency, Isolation, Durability) garantisi sunan Delta Lake formatında Gold katmanına kaydedilir. Delta Lake tercihi, veri yazılırken oluşabilecek kesintilerin tabloyu bozmamasını sağlar.

5. MAKİNE ÖĞRENMESİ VE MODEL REKABETİ

Gold katmanındaki rafine veri, çoklu model karşılaştırma (ml_training.py) aşamasına girer. Proje kapsamında beş farklı sınıflandırma algoritması eğitilmiştir: Logistic Regression, Decision Tree, Random Forest, Naive Bayes ve Gradient Boosted Trees (GBT). Özellikle GBT modeli, çok sınıflı (1-5 yıldız) bir problemde çalışabilmesi için OneVsRest(OvR) mimarisiyle sarmalanmıştır.

Her bir modelin performansı; Accuracy, F1-Score, Precision, Recall ve AUC-ROC metrikleri üzerinden ölçülür. Burada sadece doğruluk (accuracy) değerine bakılmaz; sınıflar arasındaki dengesizliği en iyi yöneten F1-Skor metriği şampiyon modeli belirlemede ana kriterdir. MLflow entegrasyonu sayesinde her deneyin parametreleri ve metrikleri merkezi bir panelden izlenebilir hale getirilmiştir.

6. SONUÇLAR, ZORLUKLAR VE GELECEK FİKİRLERİ

Proje sürecindeki en büyük teknik zorluk, 55 GB verinin yerel donanım üzerindeki disk ve I/O yükü olmuştur. Bu sorun, Spark'ın bellek yönetimi (Memory Management) ve veri örnekleme (Sampling) stratejileri ile aşılmıştır. Sonuçlar göstermiştir ki, verified_purchase ve review_word_count özellikleri, yıldız puanını tahmin etmede en etkili değişkenlerdir.

Eğitilen en iyi model, MLflow Model Registry'ye kaydedilerek üretim (production) aşamasına hazır hale getirilmiştir. Bu çalışma; verinin Kafka ile akışından, Spark ile işlenmesine ve MLflow ile tescillenmesine kadar, gerçek bir veri mühendisliği projesinin tüm gerekliliklerini başarıyla karşılamaktadır. Gelecekte bu yapıya, metin verilerinin derinlemesine analizi için BERT veya Word2Vec gibi derin öğrenme (Deep Learning) katmanları eklenerek tahmin başarısı daha da yukarı çekilebilir.

PROJE SÜRECİNDE KARŞILAŞILAN TEKNİK ZORLUKLAR VE ÇÖZÜMLER

1. Donanım ve Disk Alanı Darboğazı (En Temel Engel)

Zorluk: Projenin ham maddesi olan Amazon veri seti tek başına ~55 GB yer kaplamaktadır. Bunun üzerine her bir servis (Kafka, Spark, ZooKeeper, Jupyter) için ayağa kalkan Docker imajları ve bu imajların çalışma anında tükettiği bellek (RAM) eklendiğinde, yerel geliştirme ortamında ciddi bir donanım yetersizliği baş göstermiştir. Özellikle Silver ve Gold katmanlarına veri yazılırken I/O (Giriş/Çıkış) hızının düşmesi, sistemin kilitlenmesine neden olmuştur.

Çözüm: Bu sorunu aşmak için iki aşamalı bir strateji izlenmiştir. İlk olarak, Docker imajları "slim" versiyonlara çekilerek gereksiz kütüphanelerden arındırılmıştır. İkinci ve en kritik adım olarak, veri seti üzerinde stratejik örnekleme (sampling) yapılmıştır. Tüm boru hattı (pipeline) önce küçük bir veri kümesiyle test edilmiş, ardından Spark'ın konfigürasyon ayarlarını yerel donanıma göre optimize edilerek 55 GB verinin parça parça (mini-batch) işlenmesi sağlanmıştır.

2. Sınıf Dengesizliği (Imbalanced Data) Problemi

Zorluk: Keşifsel Veri Analizi (EDA) aşamasında fark edilmiştir ki; Amazon kullanıcılarının %70'inden fazlası 5 yıldız vermektedir. Bu durum, modelin ezberlemesine (overfitting) ve azınlıkta kalan 1 veya 2 yıldızlı yorumları tahmin edememesine yol açmıştır. Model, her şeye 5 yıldız diyerek yüksek Accuracy elde ediyor ancak gerçek bir başarı göstermiyordu.

Çözüm: Sadece Accuracy (Doğruluk) değerine bakmak yerine, hassasiyeti daha iyi ölçen F1-Score ve Precision-Recall metriklerine odaklanılmıştır. Model eğitimi sırasında Random Forest gibi dengesiz verilere daha dirençli algoritmalar tercih edilmiştir.